

```
1 import tensorflow as tf
2 data = tf.constant([[44.1, 44.2], [44.3, 44.4]])
3 data = tf.reshape(data, [1, 2, 2, 1])
4 max_pool = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))
5 print(max_pool(data))
```

tf.Tensor([[[[44.4]]]], shape=(1, 1, 1, 1), dtype=float32)

Figure 1: The effect of a 2D Max Pooling layer

```
1 from tensorflow.keras import layers
2 model = tf.keras.Sequential()
3 model.add(layers.Conv2D(10, 1, input_shape=(4, 10, 10)))
4 print(model.output_shape)
5 model.add(tf.keras.layers.Flatten())
6 print(model.output_shape)
```

(None, 4, 10, 10)
(None, 400)

Figure 2: The effect of a Flatten layer

example to understand how maximum pooling works. Consider the code shown in Figure 1.

Line 1 imports TensorFlow. Line 2 defines a two-dimensional tensor named *data*. Line 3 reshapes the dimension of *data* to (1, 2, 2, 1). Line 4 uses the class *MaxPooling2D* to create the object *max_pool*. Notice that the parameter *pool_size* is (2, 2). Line 5 performs the maximum pooling operation on the tensor named *data* and prints the result. Since *data* is a two-dimensional tensor, only one element will be returned by *max_pool*. Figure 1 also shows the output of the script. It can be seen that the value returned is 44.4, the maximum value stored in the tensor named *data*.

Now, let us go back to the remaining parts of the 16th line of code in our model. The code '*layers.Conv2D(64, kernel_size=(3, 3), activation="relu")*' defines another 2D convolution layer of the neural network, but this time the parameter *filters* is specified as 64. Similarly, the code '*layers.MaxPooling2D(pool_size=(2, 2))*' again generates a maximum pooling layer. The code '*layers.Flatten()*' generates a layer that gets a multidimensional tensor as input and converts it into a single dimensional tensor. For better

understanding of the working of this layer, consider the code shown in Figure 2.

Line 1 imports the *Layers* API of Keras. Line 2 defines a sequential model. Line 3 adds a 2D convolution layer to this model by using the *add()* method. This line of code also demonstrates how a sequential model can be built layer by layer. Line 4 prints the output shape of this layer. Figure 2 shows that the output shape currently is (None, 4, 10, 10). Line 5 adds a *Flatten* layer. Line 6 prints the output shape of this *Flatten* layer. Figure 2 shows that the output shape of this layer is (None, 400). Notice that this layer does not affect the data values; it merely reshapes the data.

Now, let us consider the remaining code in the 16th line of code in our model. The code '*layers.Dropout(0.5)*' creates a *Dropout* layer. This layer randomly sets input units to 0 with a frequency of *rate* at each step during training. This helps prevent overfitting. In simple terms, overfitting is an issue faced by mathematical models when they learn too much from the training data. This results in the model not being able to make predictions based on new data. In a *Dropout* layer, input data not set to 0 is scaled up by a factor

of $1/(1 - \text{rate})$ so that the sum over all input data remains unchanged. In our model the parameter *rate* is specified as 0.5. The code '*layers.Dense(num_classes, activation="softmax")*' creates a densely connected neural network layer. The first parameter called *units* (provided by the variable *num_classes* in our example) defines the output dimension of this layer. Notice that in our model the variable *num_classes* has the value 10, because we were trying to classify the decimal digits from 0 to 9. Also, instead of '*relu*', the activation function used is '*softmax*', which is a slightly more complicated activation function. Please read the Wikipedia article titled '*Softmax function*' for better understanding of this activation function. Finally, the generated model returned by the class *Sequential* in the 16th line of code is stored in an object named *model*. So altogether, our model has seven layers. Lines 17 and 18 of our code define the batch size and the epoch size of the model.

Now, consider the 19th line of code, shown below, used in developing our model.

```
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
```

Let us try to understand the above line of code. We need to configure the training process using the method *compile()* before we start training our model. Three parameters are given to the method *compile()*. These are: loss, optimizer, and metric. Loss function is the objective that the model will try to minimise. Some of the loss functions include *mean_squared_error*, *mean_absolute_error*, *mean_absolute_percentage_error*, *mean_squared_logarithmic_error*, *hinge*, *binary_crossentropy*, *categorical_crossentropy*, *poisson*, *cosine_proximity*, etc. In our model, we have used the objective function called